GOSIM

GOSIM Shenzhen 2026

Agent First, Workflow Later

DeerFlow 的企业级智能体实践

姜宁 DeerFlow maintainer

2026-10



DeerFlow 项目核心开发

奇点智能研究院首席技术咨询师

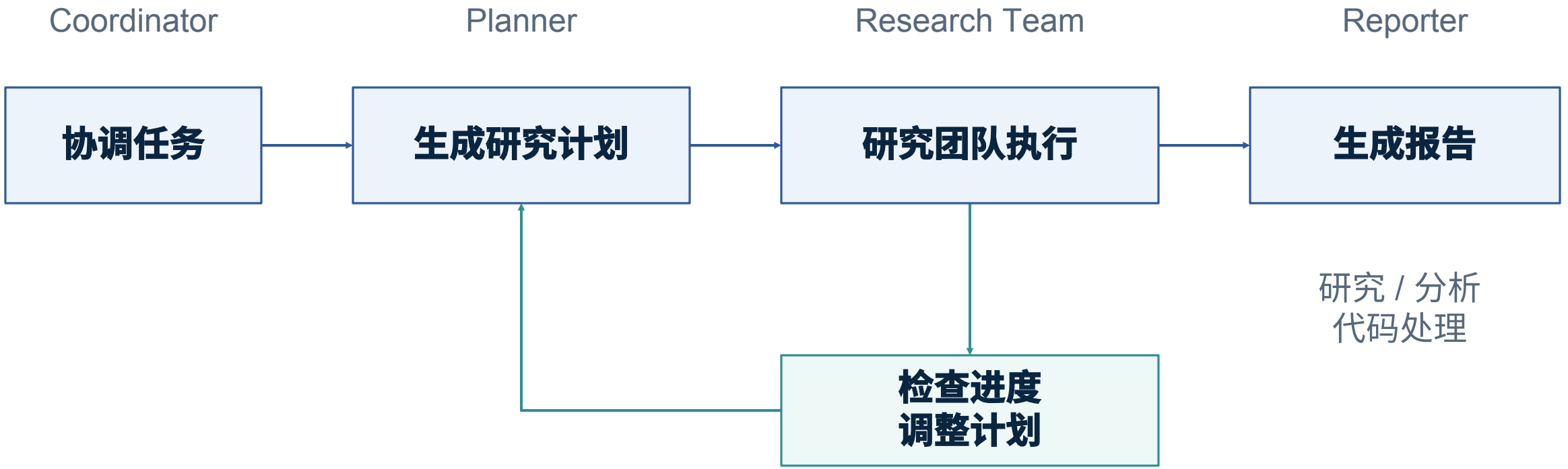
Apache软件基金会会员

前字节跳动开源首席布道师

前华为开源管理中心技术专家



DeerFlow 1.0： Deep Research 工作流



流程微调都是需要通过修改代码的方式来实现

早期企业级 Agent 应用为什么要提供完整的工作流?

模型在长任务中的不确定性

任务拆分与计划容易偏离
幻觉导致连续调用很难实现
输出质量与格式难以一致

框架提供明确的执行骨架

固定阶段与职责
明确可选择的工作路径
在交接点检查结果

模型能力是原因之一，企业应用的合规与可预测性同样重要

将复杂任务拆小，明确对应的执行步骤，降低每一步的决策难度

长时任务能力提升带来的架构变化

Workflow 预设阶段控制

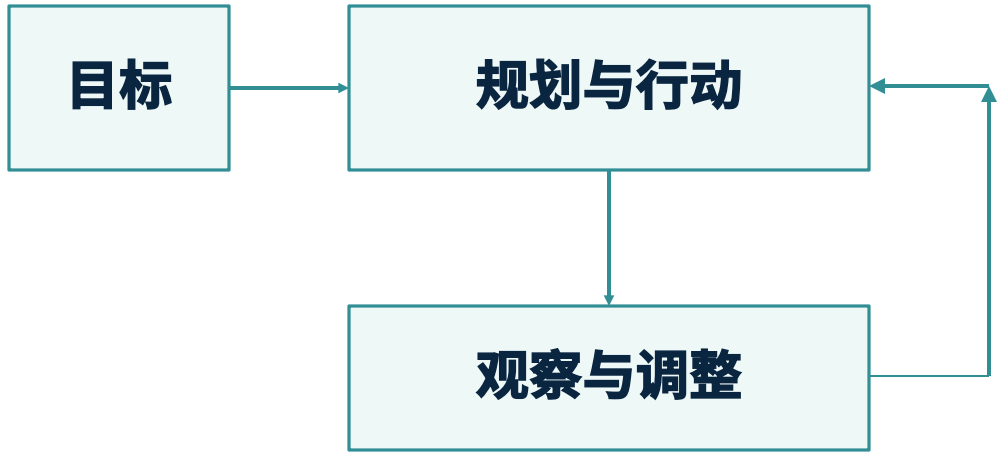
开发者提前决定任务的执行路径



模型在预设的节点中填空?

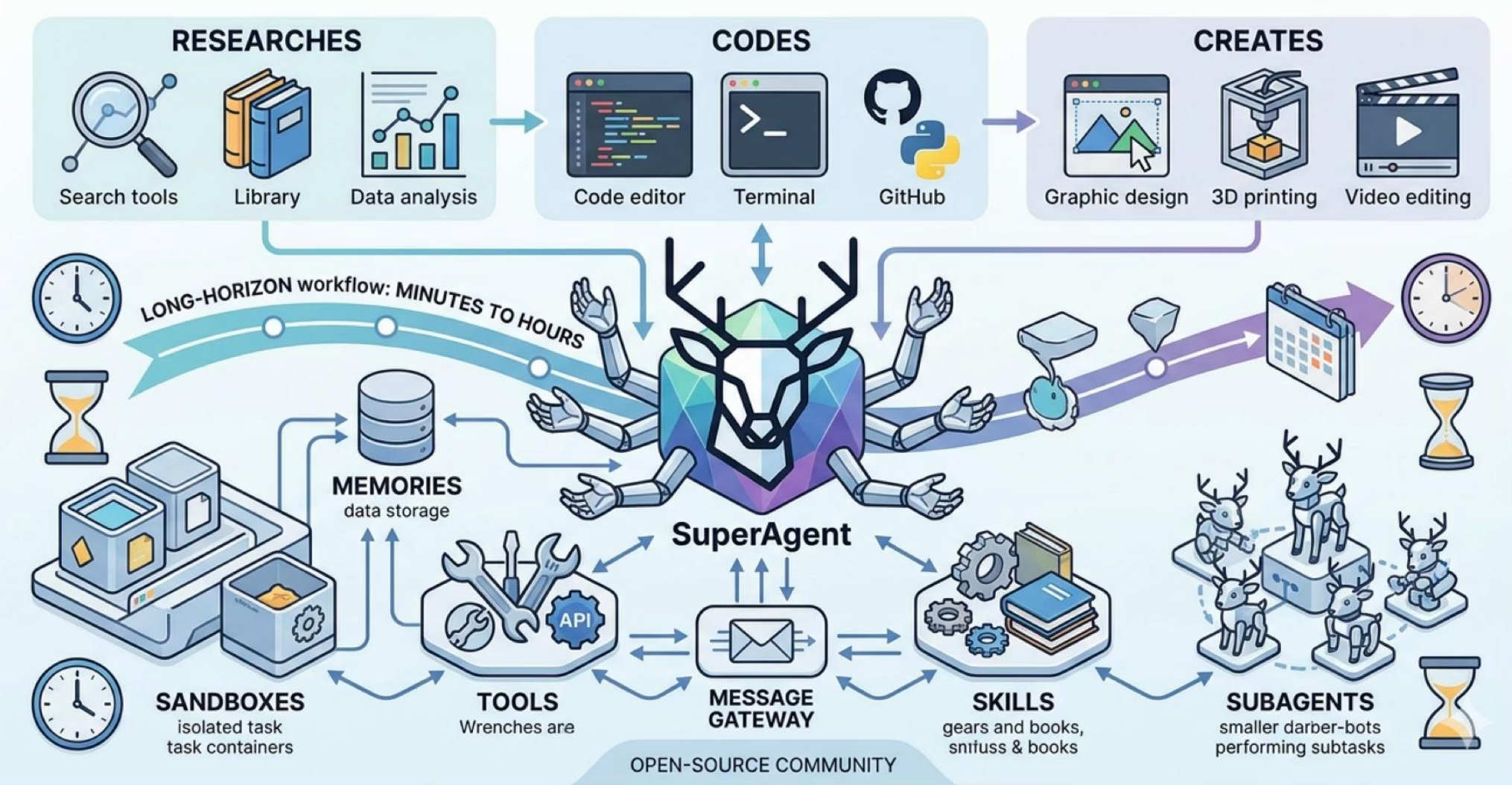
Agentic 目标驱动执行

运行时根据反馈决定下一步

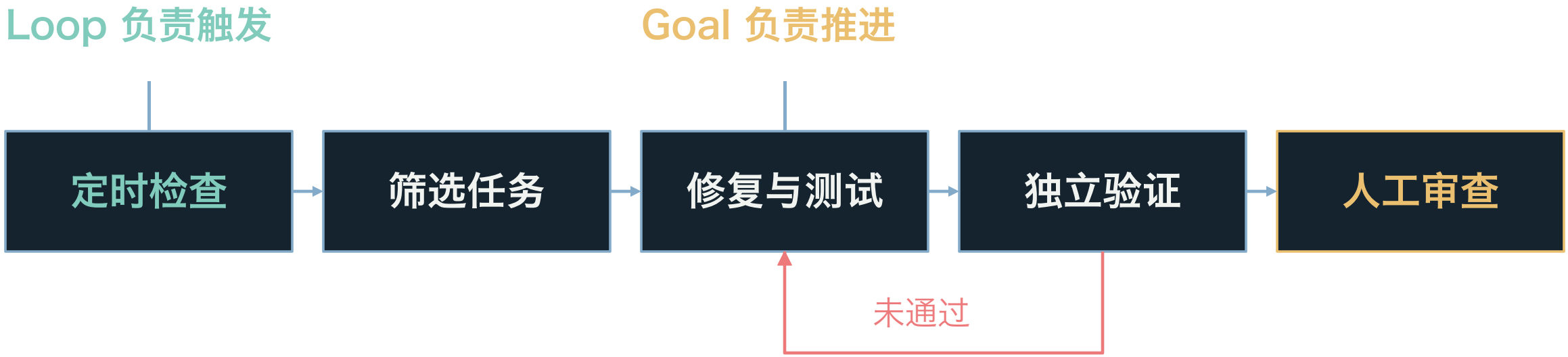


模型能力提升后，需要重新分配系统决策权

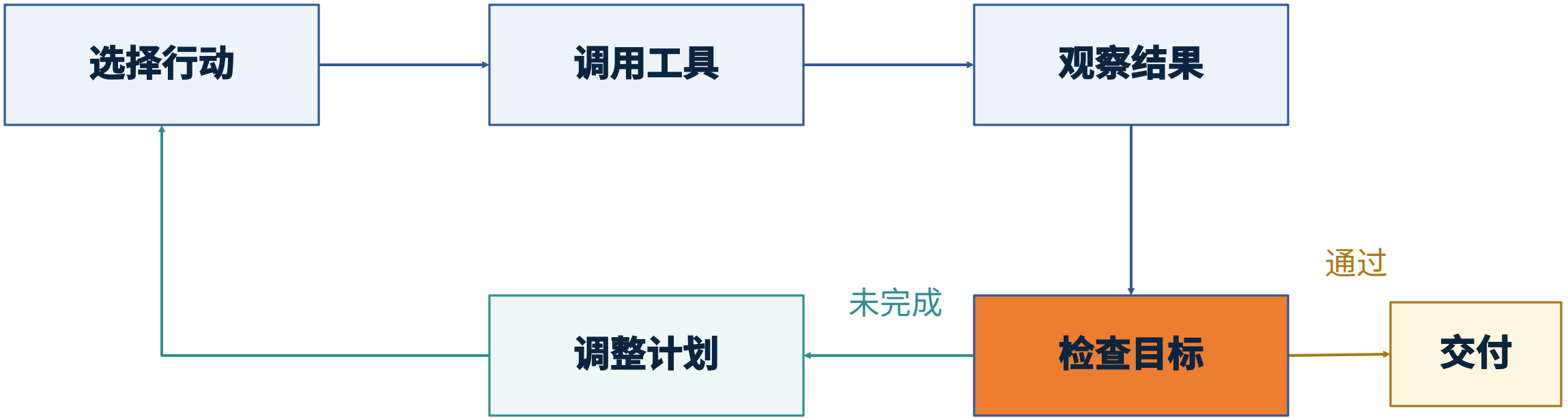
DeerFlow 2.0 超级智能体架构



Loop Engineering：持续执行循环的工程设计



Loop Engineering：持续执行循环的工程设计



运行环境　　完成标准　　提供复用的经验（记忆与 Skills）

每一轮都需要可执行的动作与可检查的反馈

运行环境：让 Agent 能够持续完成任务

工程约束：执行隔离、运行预算、中断与退出条件



模型负责提出行动，运行环境负责落实执行

完成检查：怎样证明目标已经达成？

业务目标	检查证据	判断方式
资料已收集完整	必需字段与资料清单	程序检查
工单已成功创建	系统回执与工单标识	业务系统确认
业务结果符合要求	结果内容与风险判断	必要时人工验收

未满足条件时继续补充，无法推进时退出或交接

目标完成需要证据支持，不能仅依赖模型自述



记忆与 Skills：经验如何进入下一次执行？

记忆

保留与任务相关的事实
用户偏好与历史决策
帮助恢复上下文

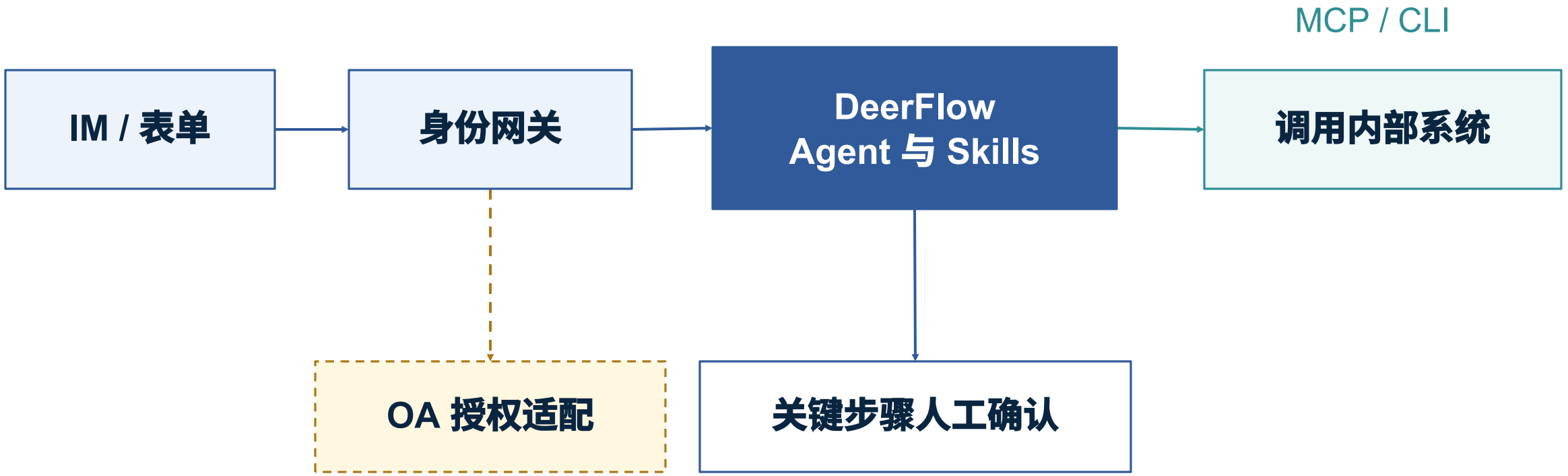
Skills

沉淀业务知识与操作方法
明确适用条件与交付要求
帮助复用可靠做法

经验入库前：筛选、验证、脱敏，并保持权限隔离

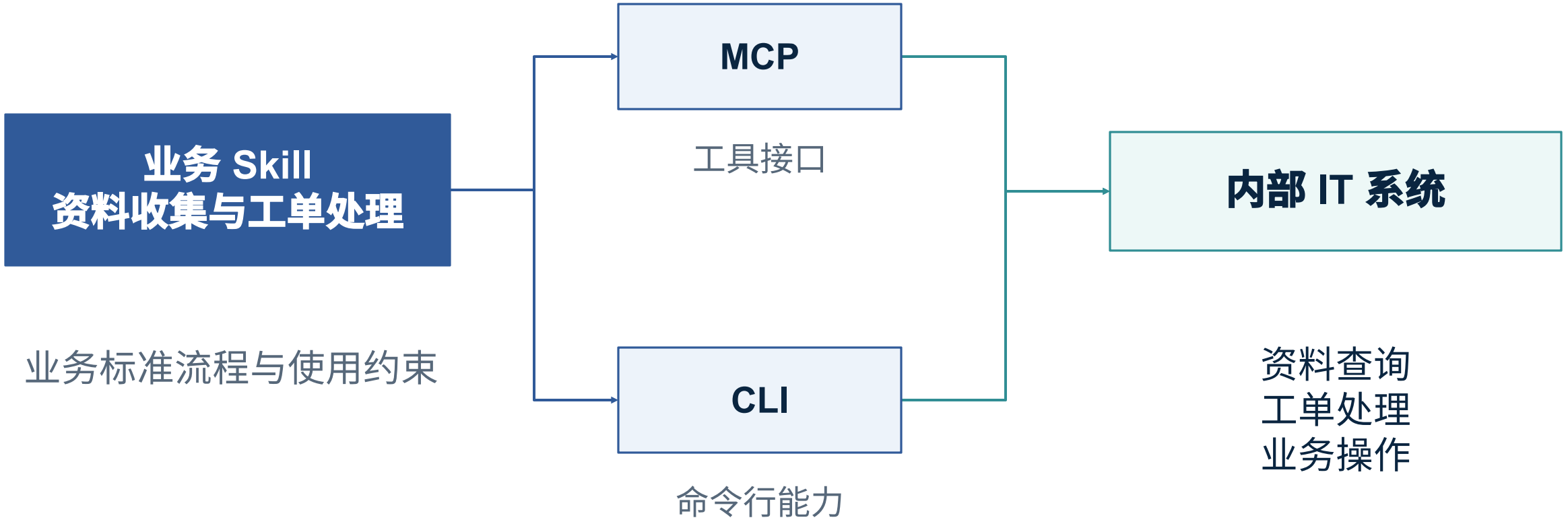
一次成功提供线索，多次验证才支持复用

DeerFlow 企业集成架构



Agent运行平台需要融入企业现有权限控制体系

Skills 与内部业务系统



工具可调用，还需要具备执行该操作的授权

人工参与的三个不同问题

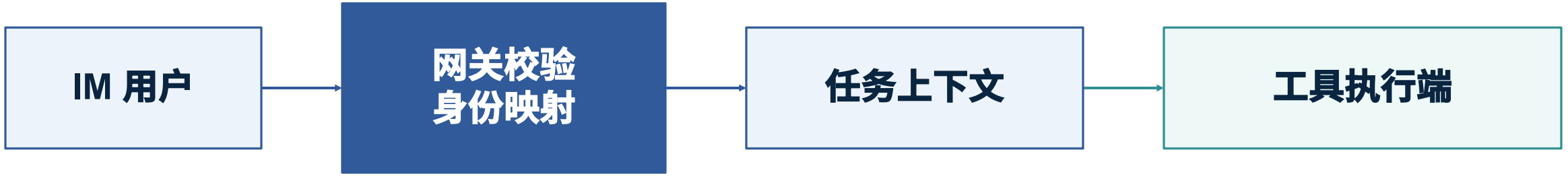
控制环节	需要回答的问题	主要作用
身份传递	谁发起、代表谁执行?	建立可信执行主体
OA 授权	允许操作哪些资源?	限定动作与访问范围
人工确认	这次具体操作是否继续?	确认、修改或拒绝

身份、授权与确认分别承担不同职责

身份传递：从 IM 用户到工具执行

企业集成设计：可信身份由系统注入，不由模型自行声明

发起人身份 **执行主体与任务关联** **执行侧核验**



发起人与审批人分别记录

审批结果关联具体任务与工具调用

身份是可信上下文，不能成为可随意改写的提示词

OA 授权与最小权限原则



资源范围

只申请当前任务所需权限
凭据留在受控执行端

操作类型

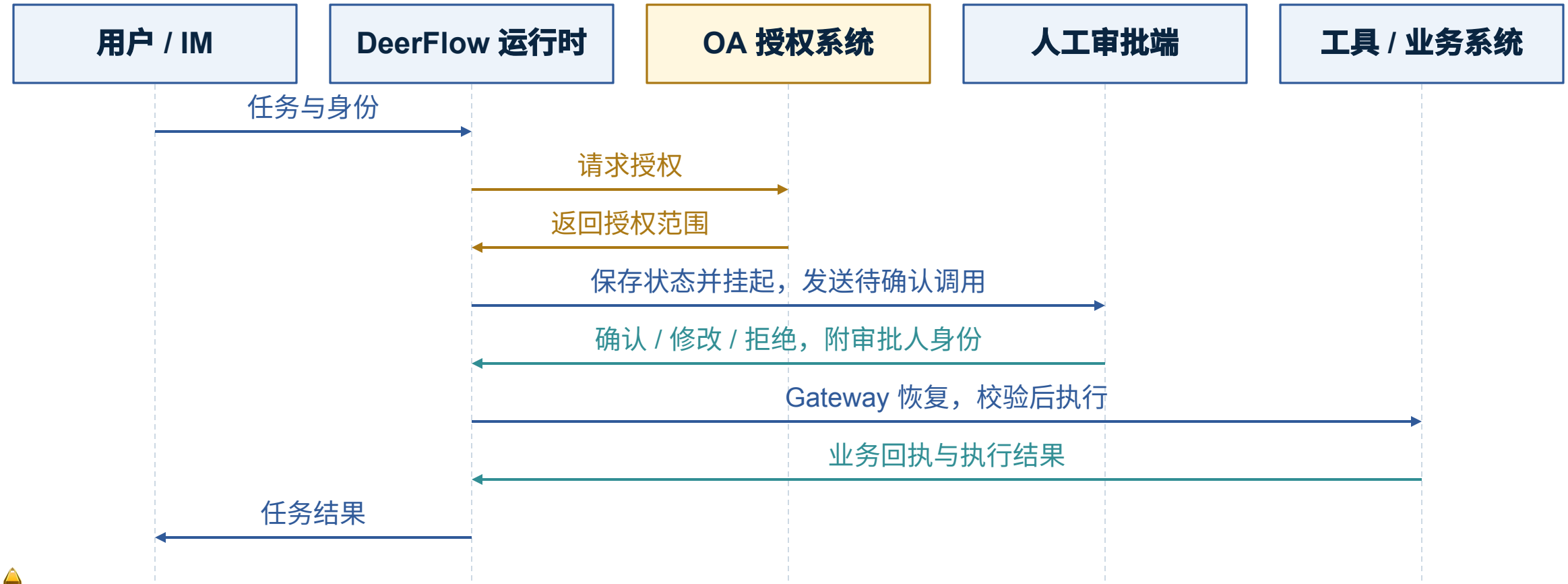
有效期限

拒绝、过期或范围不符时停止
人工同意不能替代访问控制

审批主体

授权按任务收敛，实际调用仍需执行侧检查

人工确认信息流：挂起、审批与恢复



审批决定关联原调用，恢复后继续检查业务结果

人工决定如何影响后续执行?

人工决定	后续行为	恢复前检查
同意 approve	执行原调用	身份、权限与调用关联
修改 edit	使用修改后的参数	重新校验参数及授权范围
拒绝 / 替代答复	反馈给 Agent，不执行原调用	明确拒绝原因或新信息

超时、权限失效与重复提交，都需要明确处置

恢复执行不等于跳过校验或重新执行所有步骤

用户案例：一次跨系统的业务合规巡检

根据合规要求，收集业务资料，并处理对应工单



企业合规巡检 Skills

典型任务
约 5–10 分钟

部分任务
小时级缩短至分钟级

Workflow 在 Agent 架构中的位置

架构设计建议：Agent 选择执行方式，Workflow 承接稳定子任务

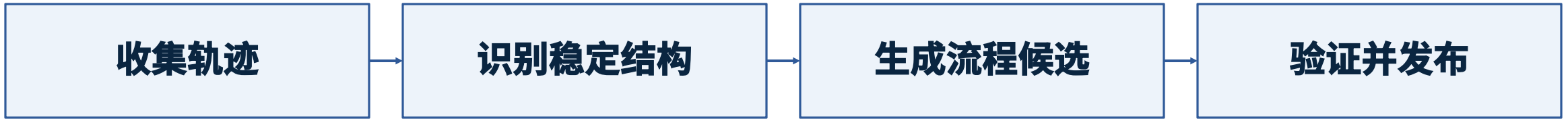
判断条件	继续由 Agent 探索	考虑固定 Workflow
业务规则	仍在变化，需要澄清	已经明确，可检查
执行路径	依赖现场判断	多次执行趋于一致
重复需求	新任务或低频尝试	高频且边界清晰

同时比较：结果质量、成功率、成本与时延

超过流程适用范围时，交回 Agent 或人工



执行轨迹如何形成可复用的 Workflow?



任务、工具调用
人工修正与结果

共同步骤
依赖与必要分支

参数化输入
代码或流程定义

测试用例与权限
人工核验与版本

流程生成需要多条轨迹与业务判断共同支撑

成功轨迹是设计证据，不能直接等同于通用流程

CodeMode：用代码表达工具调用路径

模型编写 JavaScript 组合工具调用；将流程操作沉淀为可执行代码

多轮工具交互

逐次查询、返回资料
再由模型决定后续操作

适合仍需判断的步骤

机械性处理中存在反复交互

程序化执行候选

并行查询独立数据源

↓

筛选、去重与完整性检查

↓

只返回所需结果与待确认动作

临时脚本还需测试与版本管理，才能成为可复用流程

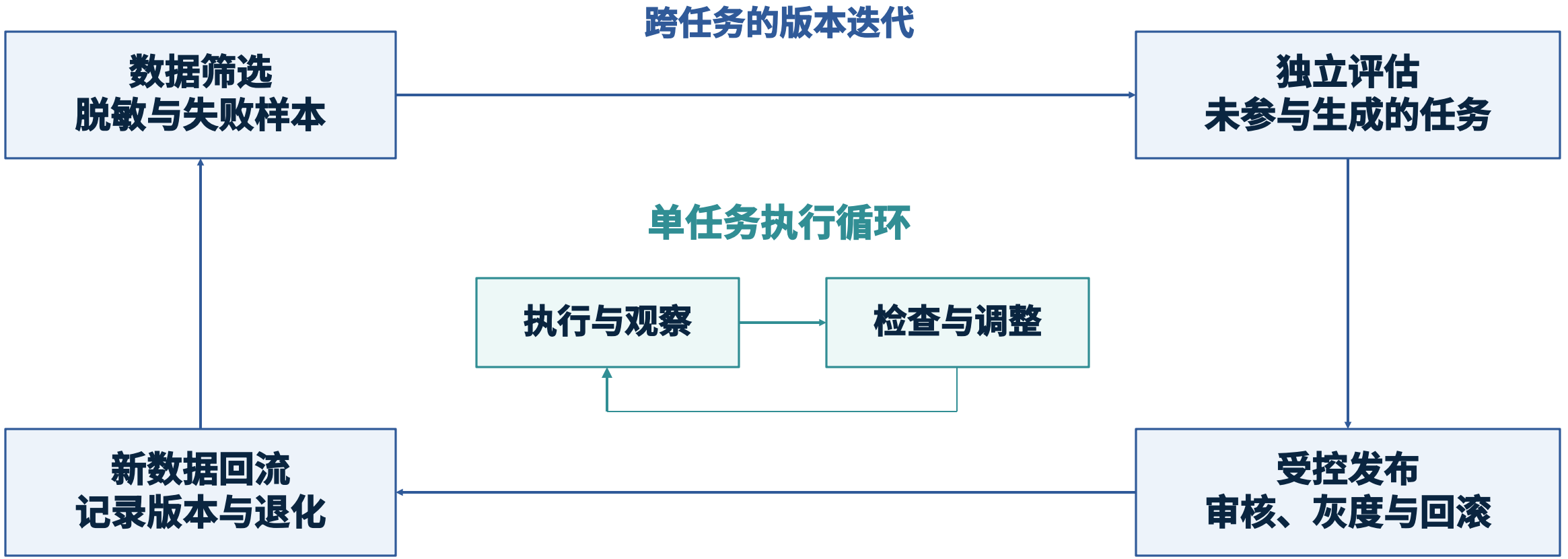
执行数据驱动的路径优化

数据输入：成功与失败轨迹、人工修正、运行指标

执行数据中的现象	可提出的优化候选
重复查询相同资料	合并查询，或在权限与时效允许时复用
多个查询没有依赖	将独立查询调整为并行执行
经常遗漏信息或需要修正	增加输入检查，完善 Skill 的要求
同类任务反复执行相同步骤	提取流程，并明确适用范围

先保证质量与授权，再优化成本和时延

数据自进化需要可验证的发布闭环



改进必须经过验证，发布后也需要可追踪与可回滚

GOSIM

GOSIM Shenzhen 2026

先获得执行经验
再固化稳定路径

运行环境与完成检查支持持续执行

身份、授权与人工确认约束具体操作

记忆与 Skills 保存可复用经验

执行数据为流程沉淀与优化提供依据

Q&A github.com/bytedance/deer-flow



